LangChain 2-Week Learning Roadmap

Absolute Beginner to Portfolio Ready | Time Commitment: 1.5 - 2 Hours/Day (6 Days/Week)

1. Course Overview

Core Objective	LangChain ki bunyadi samajh haasil karna, LLMs ko code se connect karna, aur functional AI applications (RAG tools) banana.
Prerequisites & Tools	Basic Python (functions, dictionaries), OpenAI API Key (ya open-source alternative), VS Code / Jupyter Notebook.
Student Transformation	2 hafte mein ek aisa AI portfolio project tayar hoga jo local files/PDFs se Q&A kar sake aur GitHub par host hone ke laiq ho.

2. Week-by-Week & Day-by-Day Breakdown

Week 1: LangChain Foundations & Model I/O (Milestone: Dynamic LLM Chains & Output Parsing)

Day	Core Topic	Learning Activity	Hands-on Task	Verification
Day 1	Setup & API Config	`pip install langchain openai`, `.env` file configuration	Basic script jo LLM ko prompt bheje aur output print kare	No-error output in terminal
Day 2	Prompt Templates	`PromptTemplate` aur dynamic inputs seekhna	User input par base karne wala poem/joke generator	Dynamic response on parameter change
Day 3	Output Parsers	`PydanticOutputParser` & `StringOutputParser`	Product review se structured JSON format extract karna	Valid JSON object output
Day 4	Chat Models & Persona	ChatOpenAI, System, Human & AI messages	Specific persona-based chatbot assistant (e.g. Sarcastic Coder)	Persona instructions followed correctly
Day 5	LCEL (Chains)	Pipe operator (`|`) se components chain karna	Prompt + Model + Parser ko ek single chain mein execute karna	Single `chain.invoke()` call successful
Day 6	Mini-Project 1	Week 1 concepts integration	Automated Blog Outline Generator banana aur GitHub par push karna	Clean GitHub code & README
Day 7	Buffer & Review	Code refactoring & bug fixing	Code comments add karna aur concepts solidify karna	All Week 1 tasks working smoothly

Week 2: Retrieval-Augmented Generation (RAG) & Portfolio Capstone (Milestone: PDF Q&A Bot)

Day	Core Topic	Learning Activity	Hands-on Task	Verification
Day 8	Doc Loading & Splitting	`PyPDFLoader`, `RecursiveCharacterTextSplitter`	PDF load karke 1000-char chunks mein break karna	Text chunks list printed in console
Day 9	Embeddings & Vector Store	OpenAI Embeddings & ChromaDB / FAISS	Text chunks ko vector database mein store karna	Vector DB created without error
Day 10	Retrieval & Search	Similarity search & Vector Retriever	User query ke mutabiq exact PDF paragraph retrieve karna	Relevant chunk fetched correctly
Day 11	RAG Chain Integration	`create_retrieval_chain` architecture	Context + Query LLM chain setup for document Q&A	Answers restricted to PDF data
Day 12	Memory Integration	`RunnableWithMessageHistory`	Chat history context add karna multi-turn chat ke liye	Follow-up questions answered accurately
Day 13	UI Integration (Streamlit)	Streamlit python library basics	Simple web UI banana for PDF upload & chat interface	Web app running on localhost
Day 14	Portfolio & Deployment	Project documentation & GitHub setup	Professional `README.md`, `requirements.txt` & final push	Public repository ready to share

3. Common Pitfalls & Solutions

- **API Key Leakage:** Key ko direct script mein hardcode mat karein. Hamesha `.env` file aur `.gitignore` ka istemaal karein.
- **Bad Chunking in RAG:** Document ko bilkul chunk na karna ya bohat baray chunks rakhna. Hamesha `chunk_size=1000` aur `chunk_overlap=200` se start karein.
- **Over-engineering Early On:** Shuru mein hi complex agents banana. Pehle simple LCEL chains aur single-purpose tools par grip mazboot karein.

4. Assessment & Final Deliverable

Capstone Project: "PDF Q&A Smart Assistant"

Deliverable: Ek complete Streamlit web app jahan user apni custom PDF upload kar ke direct chat kar sake. Code clean ho, GitHub par host ho, aur `README.md` mein installation guide aur demo screen recording/screenshots shamil hon.